期中考｜Python 與 NumPy 運算逐項解析

小抄只放「看了就想得起來」的結果，這份補上每一項**怎麼來的**。對應 L2 Python Basics 與 L3 NumPy。 所有結果都以 Python 3 / NumPy 實測過。**考前複習用，不帶進考場。**

A　Python 基礎

A-1　除法家族：三個符號三種行為

式子	結果	怎麼來的
`7/2`	`3.5`	`/` 是**真除法**，永遠回 float。整除也一樣：`4/2` 是 `2.0` 不是 `2`
`7//2`	`3`	`//` 是**地板除法**：算出 3.5 後往**負無窮**方向取整
`-7//2`	`-4`	−3.5 往負無窮取整是 **−4**。**不是 −3**——地板除法不是「砍掉小數」
`7%3`	`1`	餘數。配合 `//` 看：`7 = 3×2 + 1`
`-7%2`	`1`	Python 的 `%` 結果符號**跟除數走**。由 `-7 = 2×(-4) + 1` 得 1，**不是 −1**
`2**3`	`8`	`**` 是次方，不是 XOR。（`^` 才是位元 XOR，本課投影片沒教）

口訣：`//` 與 `%` 恆滿足 `a == (a//b)*b + a%b`。先定 `//` 往下取整，`%` 就被決定了。

A-2　序列的 `*` 與 `+`：是重複與接合，不是算術

式子	結果	怎麼來的
`"3"*3`	`'333'`	str × int ＝把字串**重複 3 次**再接起來
`[1,2]*2`	`[1,2,1,2]`	list 整串重複，**不是逐元素乘**。這是和 NumPy 最大的分野
`"3"+3`	`TypeError`	`+` 對 str 是接字串、對 int 是加法，Python **不自動轉型**。要先 `int("3")+3`

> **對照記：** `np.array([1,2])*2` → `[2,4]`（逐元素），`[1,2]*2` → `[1,2,1,2]`（重複）。 兩個 list 之間做 `*` 或 `@` 都會 TypeError。

A-3　切片 `a[start:stop:step]`

以下皆用 `a = [0,1,2,3,4,5]`。規則只有一條：**含頭不含尾**，三個位置都可省略。

式子	結果	怎麼來的
`a[1:4]`	`[1,2,3]`	取索引 1、2、3；索引 4 是 stop，**不含**
`a[-2:]`	`[4,5]`	負索引從尾數：−1 是最後一個、−2 是倒數第二個；stop 省略＝到底
`a[::2]`	`[0,2,4]`	start/stop 都省略＝整段，step 2 ＝每 2 個取 1
`a[::-1]`	`[5,4,3,2,1,0]`	step 為 −1 ＝從尾往頭走，效果就是**整串反轉**
`list(range(1,10,3))`	`[1,4,7]`	從 1 每次 +3：1、4、7；下一個 10 已達 stop 故停。**不含終點**

A-4　推導式、容器與查值

式子	結果	怎麼來的
`[x**2 for x in range(5) if x%2==0]`	`[0,4,16]`	執行順序是**先迭代 → 再 `if` 篩 → 最後算前面的式子**。range(5) 給 0..4，篩出 0,2,4，各自平方
`len(set("hello"))`	`4`	字串被**逐字元**放進 set，重複的 l 只留一個 → `{h,e,l,o}`

`bool([])`	**`False`**	空容器視為假。`0`、`""`、`None`、`{}` 同理
`d={'a':1}; d.get('b',0)`	**`0`**	`get` 查不到就回**預設值**，不拋錯；沒給預設值則回 `None`
`d['b']`	**`KeyError`**	中括號查不到就**拋錯**。這組對比最常考
`t=(1,2); t[0]=5`	**`TypeError`**	tuple **不可變**，建立後不能改元素（但可以整個重新指派 `t=(5,2)`）

A-5　型別轉換與格式化

式子	結果	怎麼來的
`f"{3.14159:.2f}"`	**`'3.14'`**	`:.2f` 四捨五入到小數 2 位。**回傳的是字串**，不是數字
`int("3.5")`	**`ValueError`**	`int()` 只解析**整數字串**，看到小數點就失敗
`int(float("3.5"))`	**`3`**	要先 `float()` 再 `int()`；`int()` 對 float 是**往零截斷**（`int(-3.5)` 是 −3，與 `//` 不同）

A-6　可變預設參數：定義時只建立一次

```
def f(x, L=[]):
    L.append(x)
    return L

f(1)        → [1]
print(f(2)) → [1, 2]   ← 不是 [2]
```

怎麼來的：預設值 `[]` 在 **def 這一行被執行時**就建立好，整個程式生命週期只有這一個 list 物件。 每次呼叫沒傳 `L`，用的都是**同一個**，所以內容會累積。 正確寫法是 `def f(x, L=None): L = [] if L is None else L`。

A-7　容器怎麼選

容器	可變	有序	查詢	用途與判準
`list`	✔	✔	依位置 O(1)、找值 O(n)	要增刪、要保留順序
`tuple`	✘	✔	同 list	固定組合；元素皆可雜湊時可當 dict 的鍵
`dict`	✔	插入順序	雜湊，平均 O(1)	依名字查值
`set`	✔	✘	雜湊，平均 O(1)	去重、交集 `&` 聯集 `|`

選的時候問三件事：**要不要改**、**要不要順序**、**要不要查得快**。 `if/elif` 只執行**第一個**成立的分支——FizzBuzz 必須先判 15，否則永遠印不出 FizzBuzz。縮排本身就是語法。

B　NumPy

B-1　shape 怎麼讀

規則：`ndim` ＝中括號的層數；shape 從**最外層**往內數，每一層的長度依序排出來。

式子	shape	怎麼來的
`np.array([1,2,3])`	**`(3,)`**	一層括號、3 個元素。**一維就是一維**，既不是行向量也不是列向量
`np.array([[1],[2],[3]])`	**`(3,1)`**	外層 3 個、每個內層各 1 個 → 3 列 1 欄的直欄
`np.array([[[1],[2]]])`	**`(1,2,1)`**	三層：最外 1 個、中間 2 個、最內 1 個
`np.zeros((2,3))` / `np.ones((2,3))`	**`(2,3)`**	傳進去的 tuple 就是 shape，填滿 0 或 1
`np.eye(3)`	**`(3,3)`**	單位矩陣：對角線 1、其餘 0

B-2　乘法有三種，別混用

以下 `A=[[1,2],[3,4]]`、`B=[[5,6],[7,8]]`。

式子	結果	怎麼來的
`A*B`	`[[5,12],[21,32]]`	**逐元素**相乘，位置對位置：1×5、2×6、3×7、4×8
`A@B`	`[[19,22],[43,50]]`	**矩陣乘法**（＝`np.matmul`）：左邊的**列**點乘右邊的**欄**。左上角 = 1×5+2×7 = 19
`np.dot([1,2,3],[4,5,6])`	`32`	兩個一維陣列時就是**內積**：1×4+2×5+3×6 = 4+10+18
`(10,3,4)@(4,5)`	`(10,3,5)`	**只有最後兩維參與矩陣乘**：(3,4)@(4,5)=(3,5)，前面的 10 被當成批次原樣帶著走

> **模型裡最常見的那一行：** `z = X @ w + b`。 `X:(n,m)`、`w:(m,)` → `X@w:(n,)`，每一筆樣本得到一個分數；再加 `b`（純量，廣播到每一筆）。 反向算梯度時 `X.T @ e`：`(m,n)@(n,)` → `(m,)`，剛好是每個權重一個梯度。

B-3　`axis` 是「被壓掉」的那一維

`M = np.arange(6).reshape(2,3)` 也就是 `[[0,1,2],[3,4,5]]`，shape 是 (2,3)：axis 0 是列、axis 1 是欄。

式子	結果	怎麼來的
`M.sum(axis=0)`	`[3,5,7]`	壓掉 axis 0（列）→ (2,3) 變 (3,)。**同一欄**相加：0+3、1+4、2+5
`M.sum(axis=1)`	`[3,12]`	壓掉 axis 1（欄）→ (2,3) 變 (2,)。**同一列**相加：0+1+2、3+4+5

記法：**axis 寫幾，shape 的第幾個數字就消失**。標準化 `X.mean(axis=0)` 是「每個特徵欄各算一個平均」，正是這個道理。

B-4　兩種索引

`x = np.array([1,5,3,8])`。

式子	結果	怎麼來的
`x > 2`	`[False,True,True,True]`	比較是**逐元素**的，得到一個同長度的布林陣列
`x[x>2]`	`[5,3,8]`	**布林遮罩**：把上面那個布林陣列放進中括號，只留 True 的位置
`x[[0,3]]`	`[1,8]`	**花式索引**：把要的位置寫成清單一次取出，順序由清單決定
`np.array([1,2.5]).dtype`	`float64`	陣列元素**必須同型別**，有一個是小數就整條往 float 升
`x[:,np.newaxis].shape`	`(4,1)`	在第 1 軸插入一個長度 1 的新軸，把 `(4,)` 變成直欄。等價於 `x.reshape(-1,1)`

挑某一格的樣本就是布林遮罩的應用：`mask=(y_true==1)&(y_pred==0)` → `np.where(mask)[0]` 得到 FN 的索引。 陣列條件要用 `&` / `|` 且**各自加括號**（運算優先序的關係），不能用 `and` / `or`。

B-5　廣播：形狀不同也能運算

規則：兩個 shape 從**最右邊**對齊往左比，每一對必須**相等或其中一方是 1**；位數不夠的在左邊補 1；結果每一維取兩者較大者。

形狀	結果	怎麼來的
`(4,3) + (3,)`	`(4,3)`	右邊左補 1 成 `(1,3)`；最右 3=3 相等，左邊 1 撐成 4 → 同一列複製 4 次
`(4,3) + (4,)`	ValueError	右邊補成 `(1,4)`，最右是 3 vs 4，**兩者都不是 1** 故失敗。想成功要先 `reshape(-1,1)`
`(4,1) + (3,)`	`(4,3)`	`(4,1)` 與 `(1,3)`：兩邊各自把 1 撐開，得 4×3
`(3,1,2) – (5,2)`	`(3,5,2)`	右對齊：2=2、1 撐成 5、左邊補 1 給 3

`(8,1,6,1) + (7,1,5)`	**`(8,7,6,5)`**	右邊補成 `(1,7,1,5)`，逐位取較大者
`(n,) + (n,1)`	**(n,n)**	**最常踩的陷阱**：`(n,)` 補成 `(1,n)` 被當成一列、另一個是直欄，撐成方陣。**不報錯，但答案全錯**

B-6　距離矩陣：廣播的經典應用（L3 p43 的練習）

```
A.shape → (a, m)   # a 個點，每點 m 維
B.shape → (b, m)   # b 個點

D = A[:, None, :] - B[None, :, :]
# A[:,None,:] 是 (a,1,m)、B[None,:,:] 是 (1,b,m)
# 廣播後 D 是 (a,b,m)：每一對點在每個維度的「座標差」

dist = np.linalg.norm(D, axis=2)
# 沿 axis=2 把 m 個分量平方和開根號壓掉 → (a,b) 才是距離
```

關鍵在**插新軸的位置不同**：A 在中間插、B 在最前面插，廣播時才會「每個 A 配每個 B」。`(a,b,m)` 還不是距離，少了最後那一步 norm——小抄上寫「座標差；還不是距離」就是這個意思。

B-7　兩個容易出錯的習慣

寫法	行為	怎麼來的
`idx = rng.permutation(len(y))` `X, y = X[idx], y[idx]`	同步洗牌	先產生一組打亂的索引，特徵與標籤用**同一組** `idx` 重排，對應關係才不會錯開。分開各洗一次就毀了
`b = a`	共用	只是多一個名字指向**同一塊**資料，改 `b` 等於改 `a`
`b = a.copy()`	複製	真的另外配一份記憶體，兩者從此獨立
`np.where(z>=0, 1, 0)`	門檻分類	三參數版的 `where`：條件成立取 1、否則取 0，逐元素套用

其他性質：ndarray 元素同型別、切片出來的 view 可能不連續；`.T` 對一維陣列**無效**（一維沒有兩個軸可換），要轉直欄得用 `reshape(-1,1)` 或 `np.newaxis`； 向量化之所以快，是把 Python 迴圈換成預先編譯好的底層常式一次處理整個陣列。